Guía 2 de 3 — Sets y diccionarios

DPL1046 · Clase 3 · Para repasar en casa, a tu ritmo.

Las otras dos estructuras de hoy: el **set**, que quita repetidos, y el **diccionario**, que busca por nombre. También verás cómo se relacionan con una **tabla** y con una **API**.

---

1. El set `{ }`: los invitados distintos

A una fiesta llegan invitados y alguien anota los nombres: Ana, Luis, Ana, Pedro, Luis. ¿Qué personas **distintas** vinieron? Tres: Ana, Luis y Pedro. Los repetidos desaparecen, y no importa quién llegó primero. Eso es un **set** (conjunto).

```
invitados = ["Ana", "Luis", "Ana", "Pedro", "Luis"]
print(len(invitados))        # 5  (con repetidos)

distintos = set(invitados)   # el set quita los repetidos solo
print(distintos)             # {'Ana', 'Luis', 'Pedro'}  (el orden puede cambiar)
print(len(distintos))        # 3
```

Dos preguntas frecuentes:

- **¿Permite duplicados?** No. Si le pasas una lista con repetidos, se queda con **uno de cada valor**, sin dar error.
- **¿Los elimina solo?** Sí. Tú no haces nada: `set(...)` lo hace.

Un set **no tiene orden**: al ejecutarlo varias veces los elementos pueden salir en otro orden. No es un error.

Restar conjuntos

Si A son los invitados y B los que ya pagaron, **A menos B** son los que faltan por pagar:

```
invitados = {"Ana", "Luis", "Pedro"}
pagaron = {"Luis"}
print(invitados - pagaron)    # {'Ana', 'Pedro'}
```

En Olist

```
estado_orden = orders["order_status"].tolist()    # 99.441 estados, muchos
repetidos
estados_unicos = set(estado_orden)                # quita los repetidos
print(len(estados_unicos))                        # 8
```

Las 99.441 órdenes tienen solo **8 estados distintos**.

```
todas = set(orders["order_id"])                  # todas las órdenes
con_resena = set(reviews["order_id"])            # órdenes con reseña
sin_resena = todas - con_resena                  # la resta
print("Sin reseña:", len(sin_resena))            # 768
```

> **¿Puede una orden tener más de una reseña?** Sí. En `reviews` hay 99.224 filas pero solo **98.673 órdenes distintas**: 551 órdenes tienen reseñas repetidas. El set las cuenta una sola vez. Por eso `99.441 - 99.224 = 217` **no** da el resultado correcto, y la resta de sets da **768**.

---

2. El diccionario `{ clave: valor }`: la libreta de contactos

En la libreta de tu celular no buscas "el contacto número 37": buscas por **nombre** y aparece el teléfono. El nombre es la **clave**; el teléfono es el **valor**.

```
contactos = {"Ana": "555-1234", "Luis": "555-9876"}
print(contactos["Ana"])        # 555-1234
```

Se escribe con **llaves** y **dos puntos**: `{ clave: valor, clave: valor }`.

> **Para no confundir:** el **set** tiene llaves **sin** dos puntos `{"a", "b"}`. El **diccionario** tiene llaves **con** dos puntos `{"a": 1}`.

Los cuatro movimientos

```
traduccion = {"delivered": "Entregado", "shipped": "Enviado", "canceled":
"Cancelado"}

traduccion["delivered"]                      # 'Entregado'  (buscar por clave)
traduccion.get("invoiced", "no lo tengo")    # 'no lo tengo' (si no existe, no se
cae)
traduccion["processing"] = "En proceso"      # agregar una pareja nueva
```

Forma	Si la clave no existe
`traduccion["invoiced"]`	**Error** `KeyError`
`traduccion.get("invoiced", "no lo tengo")`	Devuelve el texto por defecto, **sin error**

Una orden como diccionario

```
orden = {"id": ids[0], "estado": estado_orden[0]}
print(traduccion[orden["estado"]])           # Entregado
```

La última línea se lee **de adentro hacia afuera**: primero `orden["estado"]` da `delivered`; después ese resultado se usa como clave en `traduccion[...]` y da `Entregado`. Una libreta que apunta a otra libreta.

Debes ver (`04_diccionarios.py`):

```
El diccionario: {'delivered': 'Entregado', 'shipped': 'Enviado', 'canceled':
'Cancelado'}
delivered: Entregado
invoiced: no lo tengo
Ahora: {'delivered': 'Entregado', 'shipped': 'Enviado', 'canceled': 'Cancelado',
'processing': 'En proceso'}
Una orden: {'id': 'e481f51c…', 'estado': 'delivered'}
En español: Entregado
```

---

3. Diccionarios, tablas y APIs

Una tabla es una planilla de Excel: **filas** y **columnas**.

nombre	edad	ciudad
Ana	30	Santiago
Luis	25	Temuco

- **Una fila es un diccionario:** `{"nombre": "Ana", "edad": 30, "ciudad": "Santiago"}`.
- **Una columna es una lista:** eso es lo que hace `.tolist()`.
- **Una tabla es una lista de diccionarios**, uno por fila.

Una API es una forma de pedirle datos a otro sistema por internet, siguiendo reglas claras. Piensa en el **garzón** de un restaurante: no entras a la cocina, le pides al garzón y él te trae el plato. Lo que te trae la API tiene forma de **diccionario**: la clave es el nombre del dato y el valor es el dato. En la Clase 4 usaremos una API real.

---

4. Las cuatro estructuras juntas

Estructura	Símbolo	Orden	Se modifica	Repetidos	Ejemplo Olist
Lista	`[ ]`	Sí	Sí	Sí	Flujo de estados de una orden
Tupla	`( )`	Sí	**No**	Sí	`(id, estado)`
Set	`{ }`	**No**	Sí	**No**	Órdenes sin reseña
Diccionario	`{ : }`	Por clave	Sí	Claves **no**	Traducción de estados

5. Práctica opcional (si quieres reforzar)

No es tarea ni se entrega.

1. Crea una lista con `["rojo", "azul", "rojo", "verde", "azul", "rojo"]`. ¿Cuántos elementos tiene? ¿Cuántos colores distintos hay?
2. Crea el diccionario `edades = {"Ana": 30, "Luis": 25}`. Muestra la edad de Luis, agrega a Pedro con 41 y busca a "Rosa" con `.get` y el texto "no está".
3. **Ficha del proyecto:** abre `05_ficha_proyecto.py`, cambia el tema, los integrantes y las tablas por los de tu pareja y ejecútalo.

▼ **Respuestas**

```
colores = ["rojo", "azul", "rojo", "verde", "azul", "rojo"]
print(len(colores))          # 6
print(len(set(colores)))     # 3

edades = {"Ana": 30, "Luis": 25}
print(edades["Luis"])                    # 25
edades["Pedro"] = 41
print(edades.get("Rosa", "no está"))     # no está
```

Vocabulario

Palabra	Qué es
Set `{ }`	Valores sin repetidos y sin orden
Diccionario `{ : }`	Parejas clave → valor; se busca por clave
Clave / valor	El nombre del dato / el dato
`set(...)`	Convierte una lista en set (quita los repetidos)
`.get`	Busca por clave y, si no existe, devuelve un valor por defecto
`KeyError`	Error de buscar una clave que no existe
Tabla	Filas y columnas; cada fila se puede ver como un diccionario
API	Forma de pedir datos a otro sistema, con reglas claras